Práctica 1: Aprendizaje Supervisado

Comparativa detallada de SVM, Árboles y Ensembles sobre el dataset de obesidad

Jordi Blasco Lozano
DNI: 74527208D
Universidad de Alicante - Escuela Politécnica Superior
Aprendizaje Avanzado - Curso 2025/2026
Email: `jbl42@alu.ua.es`

Resumen

En esta práctica se comparan varias familias de algoritmos de aprendizaje supervisado sobre un problema real de clasificación multiclase: la estimación del nivel de obesidad a partir de variables antropométricas, hábitos alimenticios y actividad física. Tras corregir el data leakage existente en la transformación Box-Cox de `Age`*, el experimento queda metodológicamente mejor planteado y también cambian algunas conclusiones. En la comparativa final generada automáticamente por el notebook, Gradient Boosting sigue ocupando la primera posición con un* ***96.45 %*** *de accuracy en test y un* ***F1 ponderado de 0.9646****, seguido por Random Forest con* ***95.73 %****, SVM-RBF con* ***94.08 %*** *y AdaBoost con stumps muy lejos del resto. Sin embargo, el análisis detallado revela dos hallazgos especialmente relevantes: el kernel lineal de SVM pasa a ser ligeramente mejor que el RBF optimizado, y un árbol individual con criterio* `entropy/log_loss` *alcanza* ***97.39 %*** *en test, rivalizando con los mejores ensembles.*

1. Introducción

El objetivo de esta práctica es aplicar y comparar varios algoritmos clásicos de aprendizaje supervisado sobre un mismo problema real, manteniendo un protocolo experimental homogéneo. La idea no es únicamente comprobar qué modelo obtiene la mejor accuracy, sino entender cómo cambian los resultados cuando aumenta la capacidad no lineal del modelo, cuando se agregan múltiples árboles o cuando se modifica la forma en que se construye la frontera de decisión.

Para facilitar la comparación con la Práctica 0, se reutiliza el dataset *Estimation of Obesity Levels Based on Eating Habits and Physical Condition* del repositorio UCI. También se mantiene la misma lógica general de limpieza y codificación, aunque en esta versión se corrige un punto metodológico importante: la transformación Box-Cox de `Age` se ajusta solo con el conjunto de entrenamiento y después se aplica a test con el mismo parámetro, evitando así cualquier fuga de información.

El problema es de clasificación multiclase con 7 categorías de peso y obesidad. La dificultad principal no está en distinguir los extremos, que suelen separarse bien con peso y altura, sino en discriminar las clases intermedias, donde pequeñas variaciones provocan fronteras más difusas.

2. Configuración Experimental

2.1. Dataset

El dataset original contiene 2111 filas y 17 columnas: 16 variables predictoras y una variable objetivo. En esta nueva versión del notebook, primero se filtran únicamente los outliers puntuales de `Weight` y `Height`, quedando 2109 instancias. A partir de ahí se realiza el split estratificado, y solo sobre train se ajusta la transformación Box-Cox de `Age` y su filtrado IQR posterior. El conjunto realmente utilizado para entrenar los modelos queda en 1686 muestras de entrenamiento y 422 de test, con 23 características tras el encoding.

Tabla 1: Características generales del dataset utilizado

Característica	**Valor**
Dataset original	2111 filas, 17 columnas
Tras IQR en Weight/Height	2109 filas, 17 columnas
Features tras encoding	23
Train/Test final	1686 / 422
Variable objetivo	NObeyesdad (7 clases)
Tipo de problema	Clasificación multiclase
Valores faltantes	0

2.2. Preprocesado corregido

El enunciado permitía no detallar el preprocesado, pero aquí conviene fijarlo con claridad porque condiciona toda la interpretación posterior. La lógica general se mantiene respecto a la Práctica 0, pero con una corrección importante para evitar leakage.

Los pasos aplicados son:

1. Eliminación mediante IQR de los outliers puntuales de `Weight` y `Height`.
2. División train/test estratificada 80/20 antes de transformar `Age`.
3. Ajuste de Box-Cox sobre `Age` exclusivamente en train, con $\lambda \approx -1{,}6770$, y aplicación posterior a test con el mismo parámetro.
4. Filtrado IQR post-Box-Cox solo en train; en esta ejecución se elimina 1 fila adicional.
5. *Label Encoding* para variables binarias y *One-Hot Encoding* con `drop_first` para variables multiclase, ajustados sobre train y alineados con test.
6. Comprobación adicional de outliers en train por Z-score; en este caso no se elimina ninguna fila adicional.

Este cambio metodológico es especialmente importante porque explica parte de las diferencias respecto a la práctica anterior. En modelos sensibles a la geometría de los datos, como SVM, una fuga de información en la transformación de una variable puede alterar de forma sutil la frontera aprendida.

2.3. Métricas

Se emplean tres niveles de evaluación:

- **Validación cruzada 5-Fold** sobre train.
- **Accuracy en test**.
- **Precision, Recall y F1 ponderados**.

Además, según la familia de modelos, se consideran medidas complementarias como número de vectores de soporte, profundidad y hojas de los árboles, OOB Score en Random Forest e importancia de características.

3. Parte 1: Support Vector Machines

3.1. Comparación inicial de kernels

Tabla 2: Comparativa inicial de SVM con kernels por defecto.

Kernel	CV Accuracy	Test Accuracy	Precision	Recall	F1-Score	N° SV	% Training
Linear	0.9496	0.9431	0.9438	0.9431	0.9431	564	33.4520
RBF	0.8576	0.8744	0.8796	0.8744	0.8758	1080	64.0569
Poly (d=3)	0.7675	0.7393	0.7496	0.7393	0.7323	1203	71.3523
Sigmoid	0.6453	0.6422	0.6402	0.6422	0.6405	1222	72.4792

El primer resultado importante es que el kernel lineal no solo sigue funcionando muy bien, sino que mejora ligeramente tras corregir el leakage: alcanza **0.9496** en validación cruzada y **0.9431** en test. Esto refuerza una idea que ya estaba latente en la práctica anterior: tras el encoding y la estandarización, gran parte del problema es separable mediante fronteras aproximadamente lineales.

El kernel **RBF** por defecto mejora algo respecto a la ejecución anterior, pero sigue claramente por debajo del lineal si no se ajustan sus hiperparámetros: **0.8576** en CV y **0.8744** en test. El kernel polinomial y el sigmoide vuelven a quedar lejos de las mejores opciones, lo que sugiere que el problema no necesita una estructura polinómica global ni encaja bien con la familia sigmoide.

También resulta muy útil observar el número de vectores de soporte. El kernel lineal utiliza **564** SV, es decir, un **33.45 %** del conjunto de entrenamiento, mientras que los kernels no lineales por defecto se mueven entre el **64 %** y el **72 %**. Esto indica una frontera efectiva bastante más compleja y un mayor coste de inferencia.

3.2. Ajuste del kernel RBF

Tabla 3: GridSearch y configuraciones representativas para SVM con kernel RBF.

C	gamma	CV Accuracy	Test Accuracy	Nº SV	% Train	Tiempo (s)
100.0000	0.01	0.9401	0.9408	632	37.4852	0.0449
100.0000	0.001	0.9282	0.9289	868	51.4828	0.0523
10.0000	0.01	0.9116	0.9242	874	51.8387	0.0447
100.0000	scale	0.9104	0.9171	814	48.2800	0.0536
100.0000	auto	0.9104	0.9171	814	48.2800	0.0494
10.0000	scale	0.9051	0.9028	858	50.8897	0.0490
10.0000	auto	0.9051	0.9028	858	50.8897	0.0497
10.0000	0.1	0.8944	0.8910	954	56.5836	0.0656
100.0000	0.1	0.8944	0.8981	941	55.8126	0.0652
1.0000	0.1	0.8754	0.8673	1043	61.8624	0.0600

El mejor punto encontrado sigue siendo $C = 100$ y $\gamma = 0{,}01$, con **0.9401** de CV Accuracy y **0.9408** de Test Accuracy. La mejora respecto al RBF por defecto sigue siendo muy grande: pasa de **87.44 %** a **94.08 %** en test.

Sin embargo, aquí aparece una diferencia importante respecto a la versión previa del notebook: una vez eliminado el leakage, el RBF optimizado ya **no supera** al kernel lineal. Queda ligeramente por debajo tanto en CV como en test. La lectura es muy interesante porque sugiere que la pequeña ventaja aparente del RBF en la versión anterior estaba probablemente inflada por el preprocesado de `Age` sobre todo el dataset.

El valor alto de C indica que el modelo se beneficia de penalizar más los errores de clasificación, mientras que $\gamma = 0{,}01$ sigue siendo lo bastante pequeño como para evitar una frontera excesivamente local. El número de vectores de soporte baja a 632, lejos de configuraciones mucho más complejas como $C = 100, \gamma = 1$, donde la frontera se vuelve casi memorística.

3.3. Ajuste del kernel polinomial

Tabla 4: GridSearch y configuraciones representativas para SVM con kernel polinomial.

degree	C	gamma	CV Accuracy	Test Accuracy	Nº SV
3	10.0000	0.1	0.8808	0.8791	792
3	1.0000	0.1	0.8731	0.8626	892
3	10.0000	auto	0.8677	0.8649	909
3	10.0000	scale	0.8677	0.8649	909
2	10.0000	0.1	0.8570	0.8341	781
2	10.0000	scale	0.8535	0.8412	898
2	10.0000	auto	0.8535	0.8412	898
2	1.0000	0.1	0.8446	0.8412	957
4	10.0000	0.1	0.8428	0.8460	862
4	1.0000	0.1	0.8404	0.8460	962

La mejor configuración polinomial pasa a ser **grado 3,** $C = 10$**,** $\gamma = 0{,}1$ con **0.8808** en validación cruzada y **0.8791** en test. Mejora respecto a la ejecución anterior, pero sigue claramente por debajo tanto del kernel lineal como del RBF. Esto sugiere que el problema no necesita interacciones polinómicas de orden alto para modelarse bien.

3.4. Implementaciones lineales

Tabla 5: Comparación entre `SVC(linear)`, `LinearSVC` y `SGDClassifier`.

Implementación	CV Accuracy	Test Accuracy	Nº SV	Tiempo (s)
SVC(linear)	0.9496	0.9431	564	0.0470
LinearSVC	0.7728	0.7796	-	0.0203
SGDClassifier	0.7189	0.6754	-	0.0197

`SVC(linear)` vuelve a ser claramente superior: **0.9496** en CV y **0.9431** en test. `LinearSVC` cae hasta **0.7796** en test y `SGDClassifier` hasta **0.6754**. La conclusión aquí no es que esas implementaciones sean malas en general, sino que en este dataset pequeño y ya bien estructurado la optimización dual exacta de `SVC` compensa de sobra.

Conviene aclarar además por qué en la tabla de implementaciones lineales solo aparece número de vectores de soporte para `SVC(linear)`. La razón no es que `LinearSVC` o `SGDClassifier` no tengan una intuición geométrica relacionada con el margen, sino que **scikit-learn no almacena un conjunto explícito de vectores de soporte en esas implementaciones**. `SVC` resuelve el problema dual y sí conserva la información necesaria para identificar qué muestras quedan activas en la frontera. En cambio, `LinearSVC` está basado en `liblinear` y trabaja principalmente con una formulación primal, mientras que `SGDClassifier(loss='hinge')` aproxima la solución por descenso estocástico sobre la pérdida hinge. En ambos casos se obtiene un hiperplano lineal, pero no un conjunto explícito de SV recuperable con `n_support_`.

3.5. Vectores de soporte

Tabla 6: Comparación del número de vectores de soporte en configuraciones representativas.

Configuración	Nº SV	% Training Set
Linear (C=1)	564	33.4520
RBF (C=10, γ=0.1)	954	56.5836
RBF (C=1, γ=scale)	1080	64.0569
Poly (d=3, C=1)	1203	71.3523
RBF (C=100, γ=1)	1268	75.2076

La pauta vuelve a ser muy clara: el kernel lineal necesita pocos vectores de soporte, mientras que RBF con γ alto y el polinomial utilizan una proporción muy elevada del training set. La configuración

extrema **RBF** ($C = 100, \gamma = 1$) usa más del **75 %** de las muestras como vectores de soporte, una señal de frontera muy local y riesgo claro de sobreajuste.

3.6. Resumen de SVM

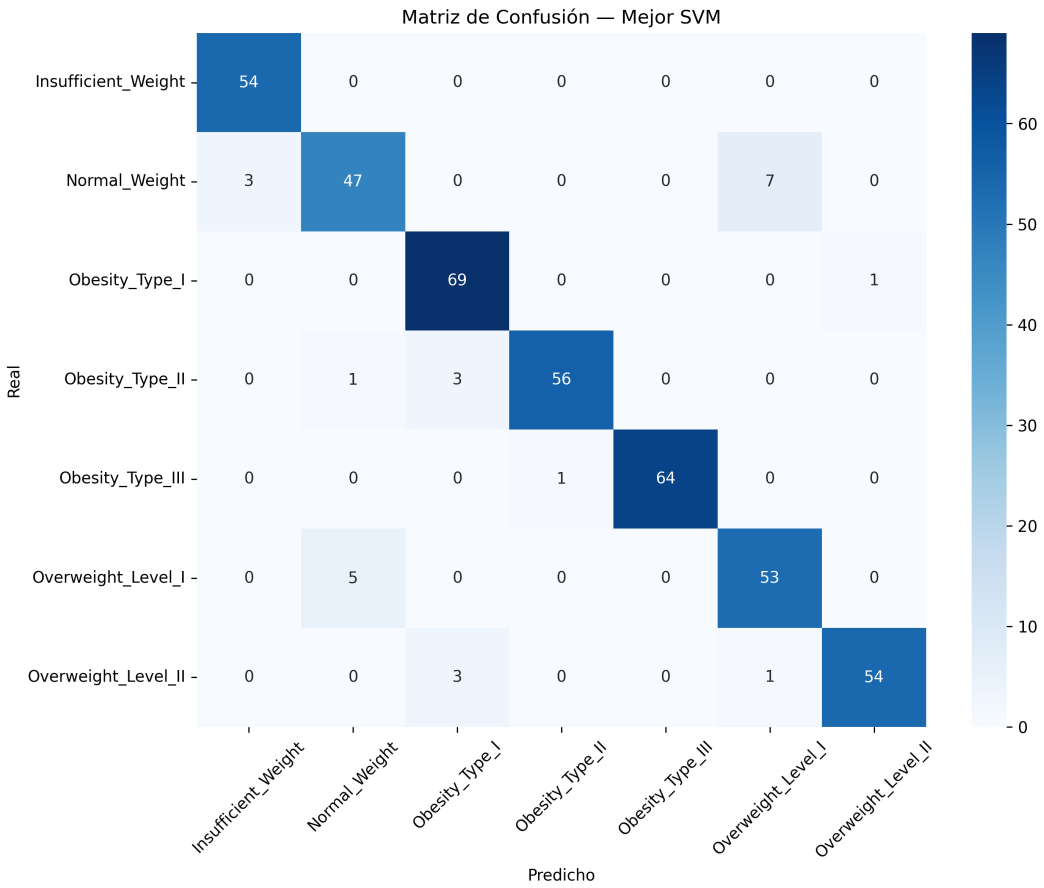

Figura 1: Matriz de confusión del mejor modelo SVM no lineal, correspondiente al kernel RBF optimizado.

Tabla 7: Resumen de los mejores resultados obtenidos por cada familia de kernel SVM.

Modelo	CV Accuracy	Test Accuracy	F1 (w)	Nº SV	% Train SV
SVM Linear	0.9496	0.9431	0.9431	564	33.4520
SVM RBF (best)	0.9401	0.9408	0.9405	632	37.4852
SVM Poly (best)	0.8808	0.8791	0.8778	792	46.9751
SVM Sigmoid	0.6453	0.6422	0.6405	1222	72.4792

Tras corregir el leakage, la lectura de esta familia cambia. El mejor modelo no lineal sigue siendo el **SVM-RBF** con **0.9408** en test y **F1 ponderado de 0.9405**, pero el **kernel lineal** pasa a estar ligeramente por delante con **0.9431** en test. La conclusión de fondo es muy valiosa: la no linealidad sigue aportando algo, pero el problema parece más lineal de lo que sugería la versión previa del notebook.

Las clases extremas siguen siendo las más fáciles de clasificar; las mayores confusiones aparecen, de nuevo, en *Normal_Weight* y *Overweight_Level_I*, donde las fronteras clínicas son más próximas.

4. Parte 2: Árboles de Decisión

4.1. Árbol sin restricciones

Tabla 8: Rendimiento del árbol de decisión sin restricciones.

Configuración	Train Accuracy	Test Accuracy	Profundidad	Nº Hojas	Tiempo (s)
Sin restricciones	1.0000	0.9265	11	97	0.0078

El árbol completo alcanza **100 %** en train pero solo **92.65 %** en test, con profundidad 11 y 97 hojas. La conclusión inmediata sigue siendo la misma: existe sobreajuste. El árbol es capaz de memorizar el entrenamiento, pero esa complejidad extra no se traduce en mejor generalización.

4.2. Efecto de la profundidad

Tabla 9: Efecto del hiperparámetro `max_depth` sobre rendimiento y complejidad del árbol.

max_depth	Train Acc	CV Acc	Test Acc	Profundidad real	Nº Hojas
1	0.2906	0.2906	0.2915	1	2
2	0.5540	0.5528	0.5427	2	4
3	0.6435	0.6340	0.6493	3	8
5	0.8523	0.8315	0.8175	5	25
7	0.9472	0.8932	0.9052	7	57
10	0.9988	0.9347	0.9265	10	95
15	1.0000	0.9389	0.9265	11	97
None	1.0000	0.9383	0.9265	11	97

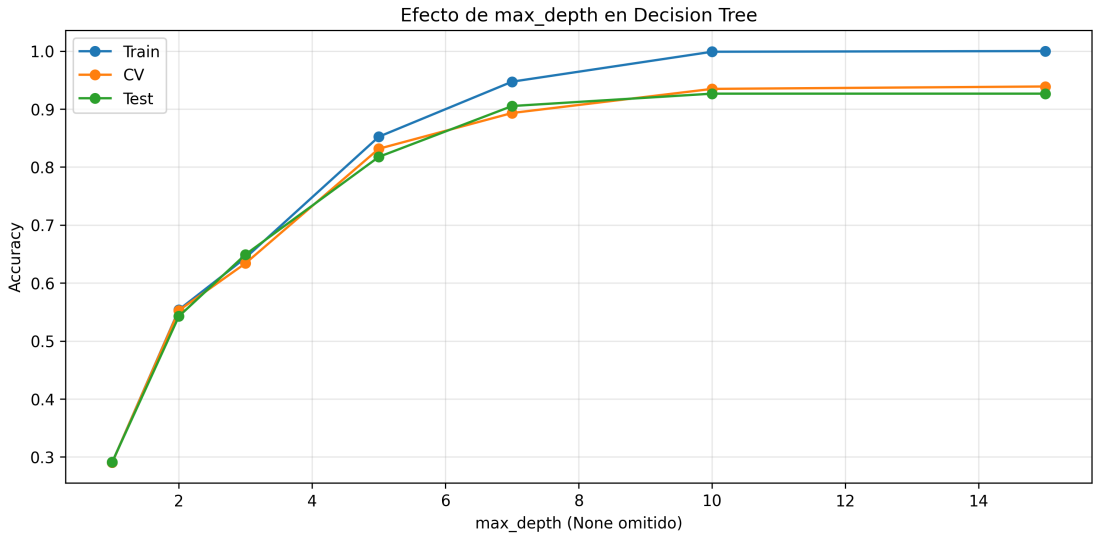

Figura 2: Curva de rendimiento del árbol de decisión en función de la profundidad máxima.

La exploración de `max_depth` vuelve a mostrar el patrón clásico. Con profundidad 1, 2 o 3 el árbol es demasiado simple y cae en *underfitting*. A medida que aumenta la profundidad, el rendimiento mejora hasta estabilizarse en torno a 10–15 niveles. Más allá de ese punto, el train se hace prácticamente perfecto y la ganancia en test desaparece.

4.3. Criterio de división

Tabla 10: Comparación entre los criterios `gini`, `entropy` y `log_loss`.

Criterio	CV Accuracy	Test Accuracy	Nº Hojas
entropy	0.9431	0.9739	81
log_loss	0.9431	0.9739	81
gini	0.9383	0.9265	97

Aquí aparece uno de los resultados más llamativos de toda la práctica. Los criterios **entropy** y **log_loss** alcanzan **0.9431** en validación cruzada y un **0.9739** en test, mientras que **gini** se queda en **0.9265**. Además, entropy/log_loss lo logran con **81 hojas**, frente a las **97** de gini.

Esto significa que, en este dataset y con el nuevo preprocesado sin leakage, el criterio de división importa mucho. La entropía está encontrando splits más informativos y convierte a un árbol individual en un competidor de primer nivel, rivalizando incluso con los mejores ensembles.

4.4. Poda previa

Tabla 11: Configuraciones más relevantes encontradas por GridSearch para poda previa.

max_depth	min_samples_split	min_samples_leaf	max_leaf_nodes	CV Acc	Test Acc
None	2	1	None	0.9383	0.9265
None	5	1	None	0.9359	0.9289
10	2	1	None	0.9347	0.9265
10	5	1	None	0.9342	0.9289
None	2	2	50	0.9306	0.9360
None	2	2	None	0.9294	0.9289
None	5	2	50	0.9288	0.9360
None	10	1	None	0.9282	0.9123
None	5	2	None	0.9282	0.9242
10	2	2	50	0.9282	0.9360

La búsqueda de poda previa no aporta una mejora clara en esta ejecución. La mejor configuración dentro del espacio explorado es, de hecho, el árbol prácticamente sin restricciones: `max_depth=None`, `min_samples_leaf=1`, `min_samples_split=2`, con **0.9383** de CV y **0.9265** en test.

Sí aparecen algunas configuraciones algo más compactas que suben el test hasta **0.9360**, pero con una CV ligeramente inferior. Lo más importante es que este grid no estaba variando el criterio de división, y precisamente ahí es donde está el mayor salto de rendimiento en la familia de árboles.

4.5. Poda posterior por cost-complexity pruning

Tabla 12: Evaluación de la poda posterior mediante `ccp_alpha`.

ccp_alpha	CV Accuracy	Test Accuracy	Profundidad	Nº Hojas
0.0009	0.9389	0.9313	11	71
0.0000	0.9383	0.9265	11	97
0.0006	0.9383	0.9289	11	80
0.0011	0.9359	0.9289	10	63
0.0012	0.9359	0.9313	9	54
0.0016	0.9264	0.9336	9	47
0.0023	0.9258	0.9384	9	40
0.0028	0.9199	0.9336	8	33
0.0042	0.9080	0.9218	8	26
0.0099	0.8849	0.8720	8	19
0.0241	0.7752	0.7607	4	10
0.0912	0.5268	0.4289	2	3

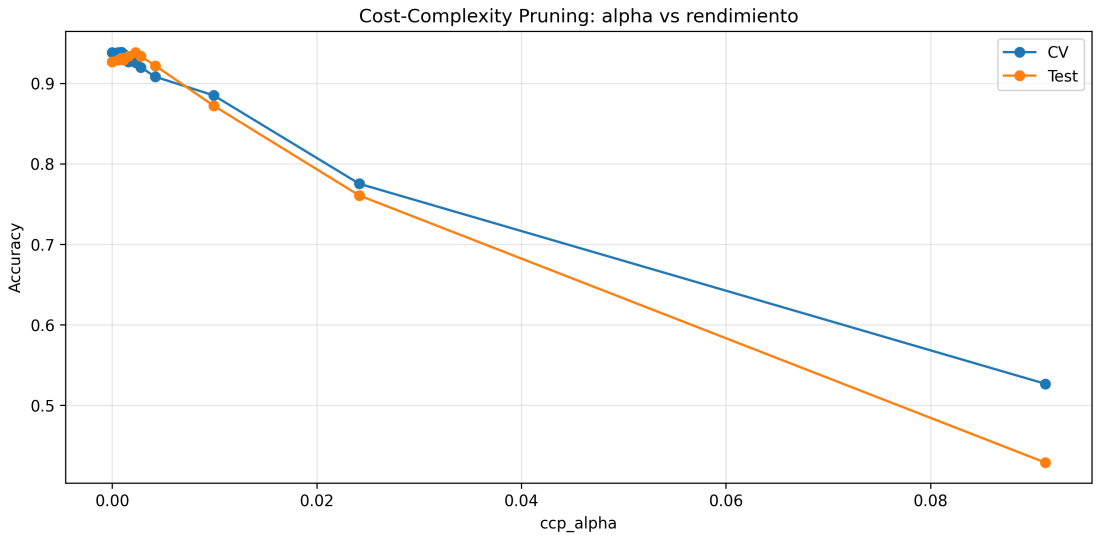

Figura 3: Relación entre `ccp_alpha` y rendimiento del árbol.

La poda posterior vuelve a ser útil. El mejor valor dentro de los alphas seleccionados es `ccp_alpha` $\approx$ 0,00089, con **0.9389** de CV y **0.9313** en test. Eso significa que una poda ligera reduce la complejidad del árbol, pasando de 97 a 71 hojas, y mejora ligeramente la generalización frente al baseline.

También aparece un detalle interesante: una poda algo más agresiva con $\alpha \approx 0{,}0023$ baja en CV, pero llega a **0.9384** en test. No conviene sobreinterpretar esa cifra aislada, pero sí sirve para recordar que una única partición train/test puede favorecer puntualmente configuraciones menos estables.

4.6. Importancia de variables

Tabla 13: Importancia de características en el árbol de decisión.

Feature	Importancia (Gini)
Weight	0.4914
Height	0.2618
Gender	0.1577
FAVC	0.0231
Age	0.0147
CH2O	0.0109
TUE	0.0094
CAEC_Sometimes	0.0082
NCP	0.0059
FCVC	0.0048
MTRANS_Public_Transportation	0.0031
CALC_Sometimes	0.0022
FAF	0.0022
family_history_with_overweight	0.0020
SMOKE	0.0014
SCC	0.0011
CAEC_Frequently	0.0000
CALC_Frequently	0.0000
CAEC_no	0.0000
CALC_no	0.0000

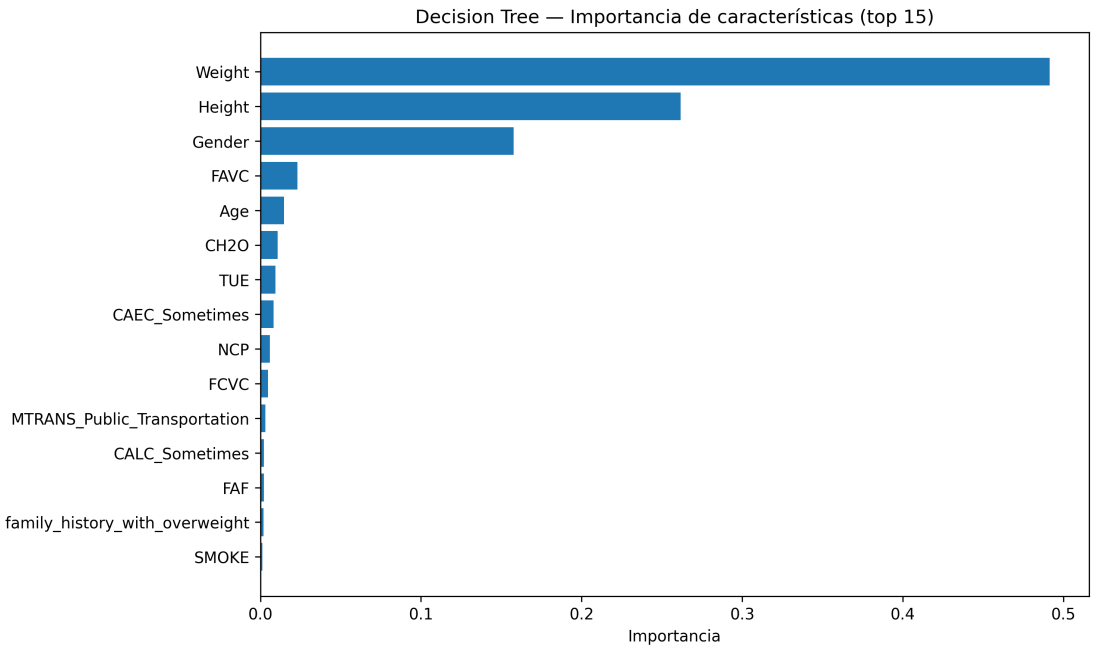

Figura 4: Importancia de características del árbol de decisión.

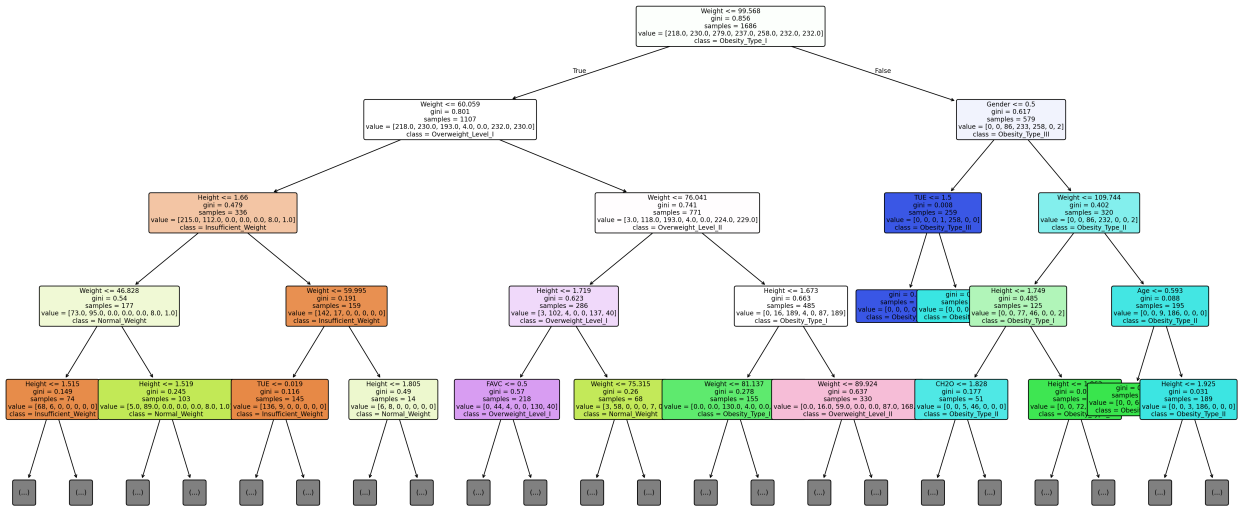

Figura 5: Visualización parcial del árbol de decisión para facilitar su lectura.

La estructura del árbol confirma, una vez más, lo que ya sugería el análisis exploratorio de la Práctica 0: **Weight** y **Height** dominan con claridad. Después aparece **Gender**, y más atrás **FAVC**, **Age**, **CH2O**, **TUE** y otras variables que refinan regiones más ambiguas del espacio.

4.7. Resumen de árboles

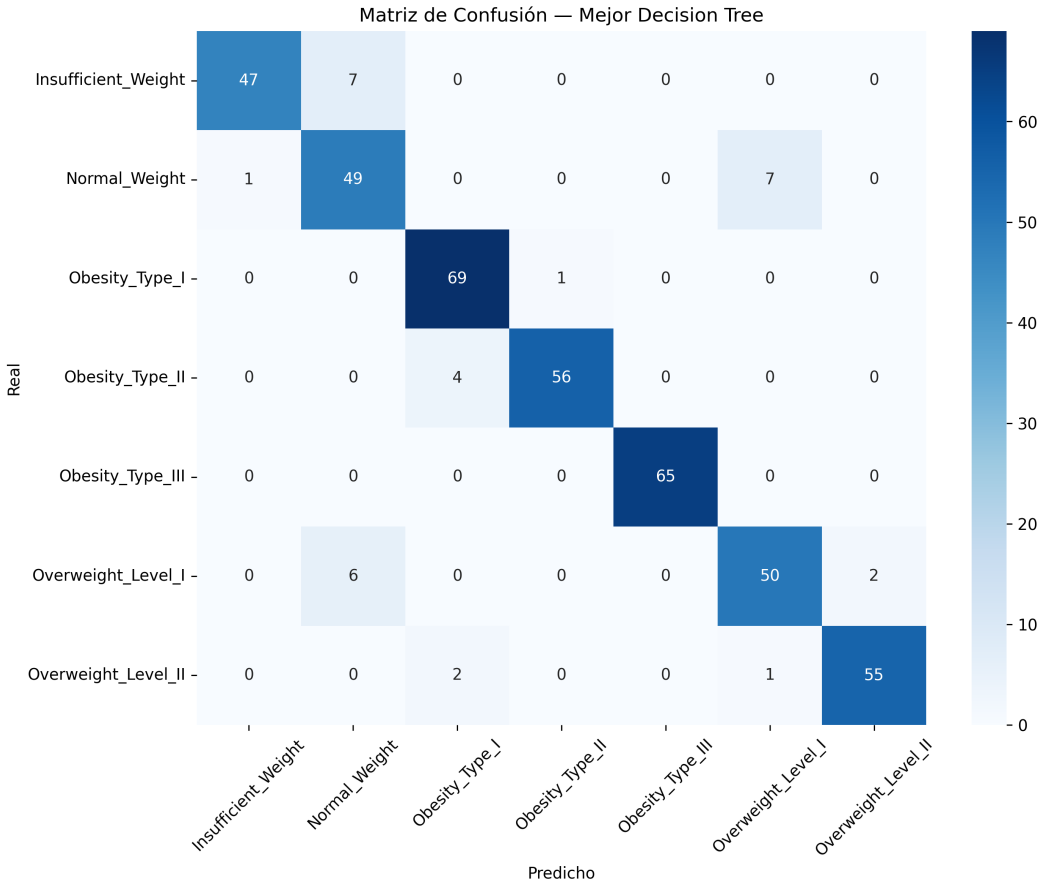

Figura 6: Matriz de confusión del árbol seleccionado en el notebook para representar esta familia.

Tabla 14: Resumen del árbol completo, la mejor poda previa y la mejor poda posterior.

Modelo	Test Accuracy	Precision (w)	Recall (w)	F1 (w)	Profundidad	Nº Hojas
Mejor poda posterior	0.9313	0.9346	0.9313	0.9318	11	71
Sin restricciones	0.9265	0.9295	0.9265	0.9271	11	97
Mejor poda previa	0.9265	0.9295	0.9265	0.9271	11	97

Si seguimos estrictamente el resumen final generado por el notebook, la familia de árboles queda liderada por la **poda posterior**, con **0.9313** en test y **F1 ponderado de 0.9318**. El árbol sin restricciones y la poda previa se quedan en **0.9265**.

Sin embargo, para que la memoria sea fiel a todos los resultados obtenidos, hay que subrayar una matización crucial: el mejor resultado aislado dentro de esta familia no está en la tabla resumen de poda, sino en la comparación de criterios, donde **entropy/log_loss** alcanzan **0.9739** en test. Es, probablemente, el hallazgo más llamativo de toda la práctica.

5. Parte 3: Random Forest y Extra Trees

5.1. Random Forest por defecto

Tabla 15: Rendimiento del Random Forest con configuración por defecto.

Configuración	OOB Score	Test Accuracy	F1 (w)	Tiempo (s)
RF por defecto (n=100)	0.9383	0.9455	0.9460	0.1486

El Random Forest por defecto alcanza **0.9455** de accuracy en test y **0.9460** de F1 ponderado, con un **OOB Score de 0.9371**. El salto respecto al árbol Gini completo sigue siendo claro: al agregar muchos árboles se reduce la varianza y se estabilizan errores que en un árbol individual dependen demasiado de muestras concretas.

5.2. Número de estimadores

Tabla 16: Efecto del número de árboles sobre OOB, test y tiempo de entrenamiento.

n_estimators	OOB Score	Test Accuracy	Tiempo (s)
10	0.8582	0.9313	0.0261
25	0.9051	0.9336	0.0546
50	0.9324	0.9408	0.0731
100	0.9383	0.9455	0.1404
200	0.9425	0.9431	0.2934
500	0.9466	0.9502	0.6247

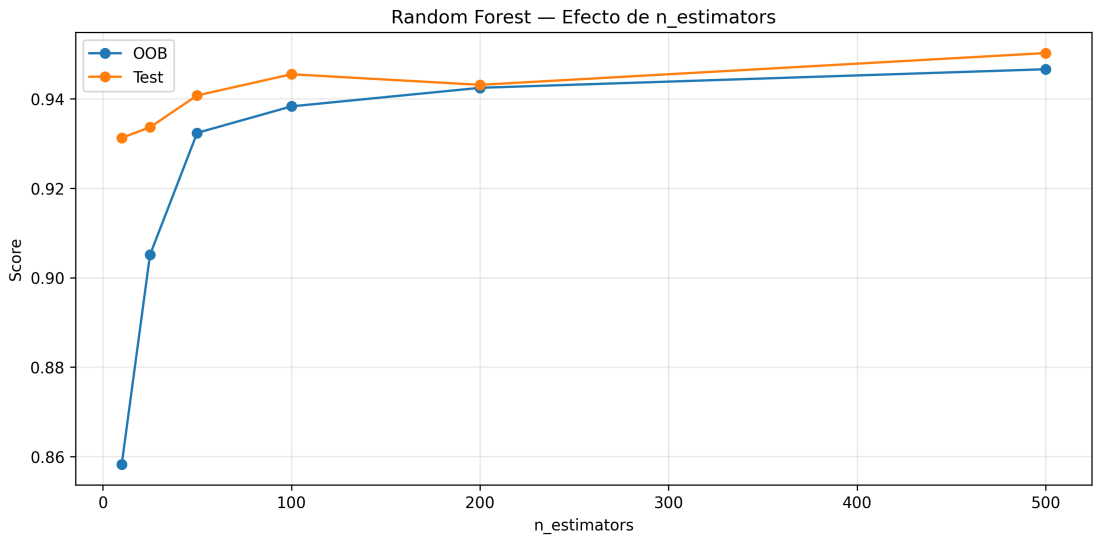

Figura 7: Curva de convergencia de Random Forest al aumentar `n_estimators`.

La tendencia es muy limpia. Pasar de 10 a 100 árboles mejora mucho el rendimiento, pero a partir de ahí aparece la ley de los rendimientos decrecientes. El test pasa de **0.9313** con 10 árboles a **0.9455** con 100, y solo sube hasta **0.9502** con 500.

El OOB Score sigue una trayectoria muy parecida y converge cerca del **94.9 %**, lo que vuelve a confirmar que es una estimación interna razonablemente fiable cuando el bosque ya tiene suficientes árboles.

5.3. Efecto de `max_features`

Tabla 17: Comparación de distintas estrategias de selección de features por split en Random Forest.

max_features	OOB Score	Test Accuracy
None	0.9573	0.9573
0.5	0.9526	0.9550
log2	0.9383	0.9455
sqrt	0.9383	0.9455

Con la nueva ejecución, usar **todas las features** (`None`) pasa a ser la mejor opción tanto en OOB como en test, alcanzando **0.9573**. La opción `0.5` queda muy cerca con **0.9550**, mientras que `sqrt` y `log2` se quedan claramente por detrás.

Esto refuerza la idea de que el problema tiene unas pocas variables dominantes, especialmente `Weight` y `Height`, y conviene que estén disponibles en todos los splits importantes. Restringir demasiado las features candidatas puede dejar fuera justo la variable más discriminante en un nodo clave.

5.4. GridSearch de Random Forest

Tabla 18: Configuraciones más competitivas encontradas por GridSearch en Random Forest.

n_estimators	max_features	max_depth	min_samples_leaf	CV Acc	Test Acc
100	None	None	1	0.9561	0.9573
100	None	20	1	0.9561	0.9573
200	None	20	1	0.9537	0.9550
200	None	None	1	0.9537	0.9550
200	None	10	1	0.9525	0.9479
100	None	10	1	0.9519	0.9550
200	None	None	2	0.9514	0.9455
200	None	20	2	0.9514	0.9455
200	None	10	2	0.9484	0.9455
100	None	20	2	0.9478	0.9502

El mejor modelo según la búsqueda efectuada es **100 árboles,** `max_features=None`**,** `min_samples_leaf=1` con **0.9561** en CV y **0.9573** en test. A diferencia de la versión anterior del notebook, ahora el barrido específico de `max_features` y el GridSearch quedan bastante alineados: ambos apuntan a `None` como mejor elección.

5.5. Importancia de características

Tabla 19: Importancia de características en Random Forest medida por impureza y por permutación.

Feature	Gini Importance	Permutation Importance	Perm Std
Weight	0.4622	0.7322	0.0226
Height	0.2536	0.3237	0.0125
Gender	0.1586	0.1690	0.0140
Age	0.0297	0.0547	0.0057
FAVC	0.0233	0.0092	0.0031
FAF	0.0113	0.0012	0.0026
TUE	0.0096	-0.0002	0.0017
NCP	0.0087	0.0012	0.0034
FCVC	0.0080	0.0024	0.0021
CH2O	0.0080	0.0012	0.0012
CAEC_Frequently	0.0046	0.0031	0.0015
CALC_no	0.0040	0.0007	0.0011
CALC_Sometimes	0.0038	0.0012	0.0022
CAEC_Sometimes	0.0038	-0.0009	0.0026
family_history_with_overweight	0.0024	0.0038	0.0016
MTRANS_Public_Transportation	0.0019	0.0000	0.0000
MTRANS_Walking	0.0017	-0.0002	0.0007
SMOKE	0.0016	0.0002	0.0007
SCC	0.0011	0.0000	0.0000
CALC_Frequently	0.0007	0.0000	0.0000
MTRANS_Motorbike	0.0004	0.0000	0.0000
MTRANS_Bike	0.0004	0.0000	0.0000
CAEC_no	0.0003	0.0000	0.0000

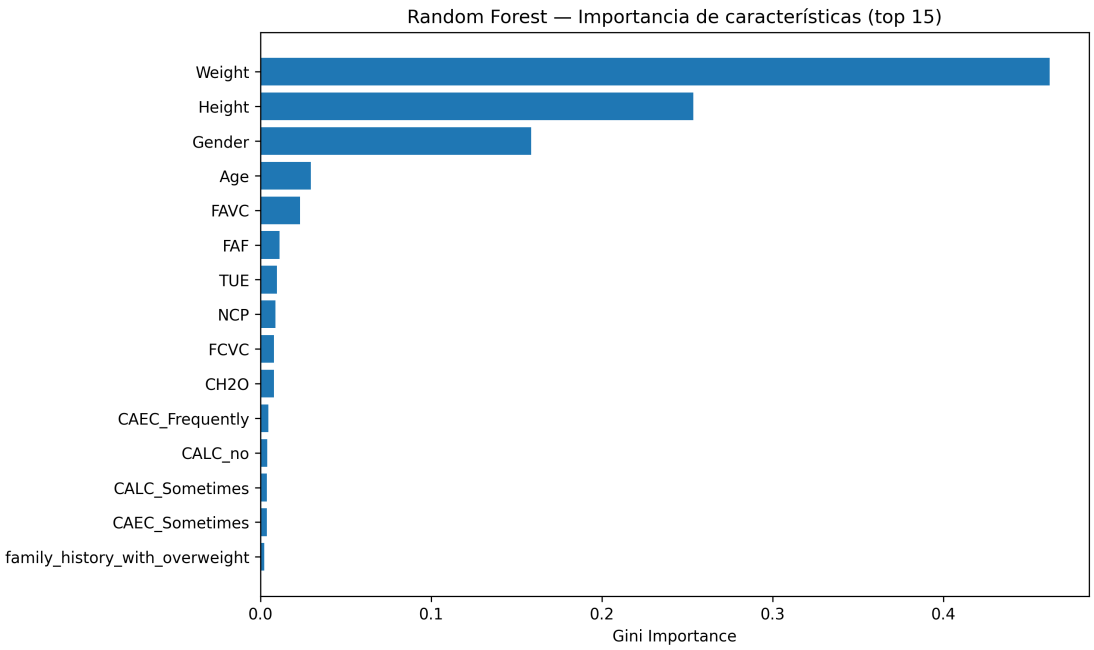

Figura 8: Importancia de características en Random Forest según reducción de impureza.

La comparación entre importancia Gini y *permutation importance* sigue contando la misma historia: **Weight** y **Height** dominan con mucha claridad, seguidas por **Gender** y **Age**. El resto de variables aportan información, pero con un peso mucho menor o más redundante.

5.6. Extra Trees

Tabla 20: Resultado de Extra Trees con 100 árboles.

Modelo	CV Accuracy	Test Accuracy	F1 (w)	Tiempo (s)
Extra Trees (n=100)	0.9187	0.9218	0.9229	0.0906

Extra Trees obtiene **0.9218** en test, por debajo tanto del Random Forest por defecto como del optimizado, e incluso por debajo del árbol post-podado. La aleatorización adicional de umbrales no ayuda aquí: introduce más sesgo del que consigue compensar con reducción de varianza.

5.7. Resumen de bagging

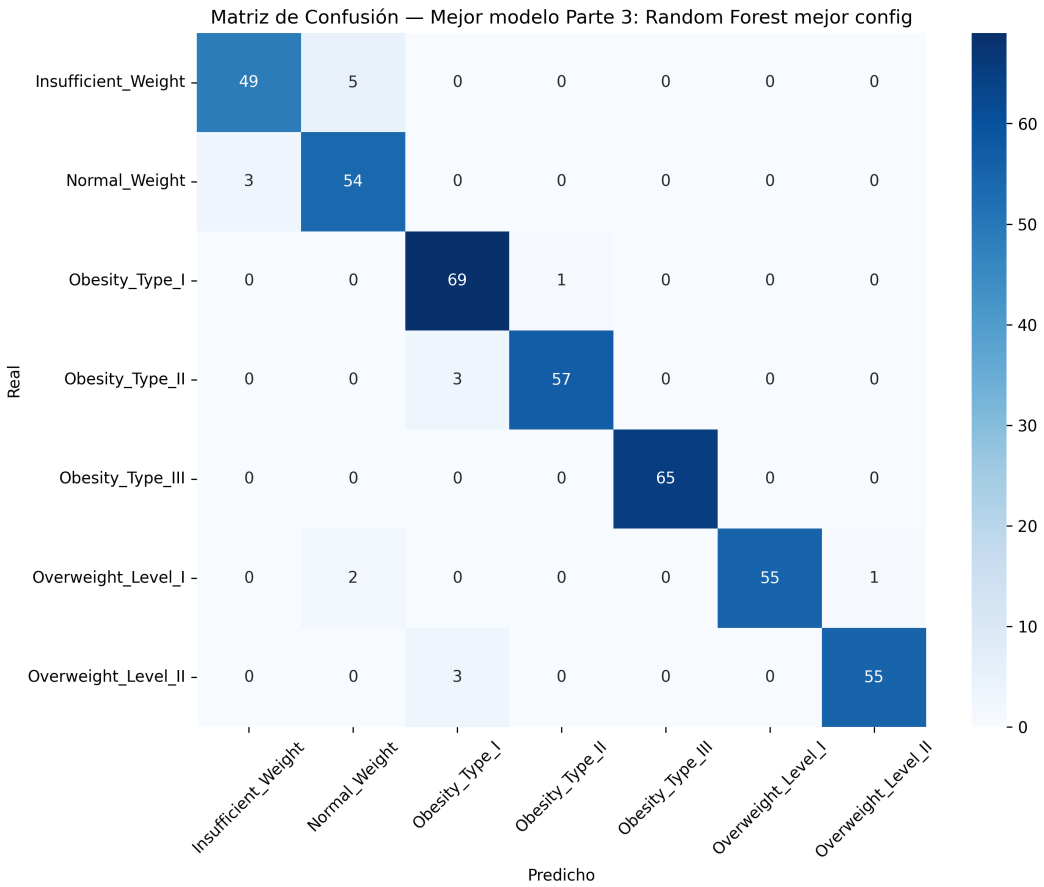

Figura 9: Matriz de confusión del mejor modelo de la familia bagging.

Tabla 21: Comparativa final de métodos basados en bagging y referencia del árbol individual.

Modelo	Test Accuracy	Precision (w)	Recall (w)	F1 (w)	OOB Score	Tiempo (s)
Random Forest mejor config	0.9573	0.9589	0.9573	0.9576	-	-
Random Forest por defecto	0.9455	0.9488	0.9455	0.9460	0.9383	-
Decision Tree (baseline)	0.9265	0.9295	0.9265	0.9271	-	-
Extra Trees	0.9218	0.9259	0.9218	0.9229	-	-

La conclusión principal sigue siendo nítida: **Random Forest** mejora de manera consistente al árbol individual Gini y alcanza **95.73 %** de test con **F1 ponderado de 0.9576**. El bagging funciona muy bien aquí porque el árbol individual ya es relativamente fuerte, pero todavía tiene suficiente varianza como para beneficiarse mucho de la agregación.

6. Parte 4: Gradient Boosting y AdaBoost

6.1. Gradient Boosting por defecto

Tabla 22: Rendimiento del Gradient Boosting con configuración por defecto.

Modelo	CV Accuracy	Test Accuracy	F1 (w)	Tiempo (s)
Gradient Boosting (default)	0.9561	0.9573	0.9579	2.0885

Con los parámetros por defecto, Gradient Boosting alcanza **0.9573** en test y **0.9579** de F1 ponderado. Sigue siendo un resultado excelente, comparable al mejor Random Forest, aunque ya no tan

claramente dominante como en la ejecución anterior del notebook.

6.2. Interacción entre `n_estimators` y `learning_rate`

Tabla 23: Relación entre número de estimadores y learning rate en Gradient Boosting.

n_estimators	learning_rate	CV Accuracy	Test Accuracy
200	0.1000	0.9620	0.9739
50	0.5000	0.9608	0.9550
100	0.1000	0.9561	0.9573
50	0.1000	0.9490	0.9336
100	0.0500	0.9460	0.9336
200	0.0100	0.9181	0.8815

La configuración con mejor resultado en test dentro de este barrido es **200 árboles con** `learning_rate = 0.1`, alcanzando **0.9739**. En cambio, `learning_rate = 0.01` con 200 árboles se queda en **0.8815**. La lectura de fondo se mantiene: un learning rate muy bajo necesita muchas más iteraciones para resultar competitivo.

6.3. Profundidad de los árboles base

Tabla 24: Efecto de `max_depth` en Gradient Boosting.

max_depth	Train Acc	CV Accuracy	Test Accuracy
5	1.0000	0.9656	0.9573
3	1.0000	0.9561	0.9573
2	0.9905	0.9425	0.9336
1	0.8826	0.8487	0.8436

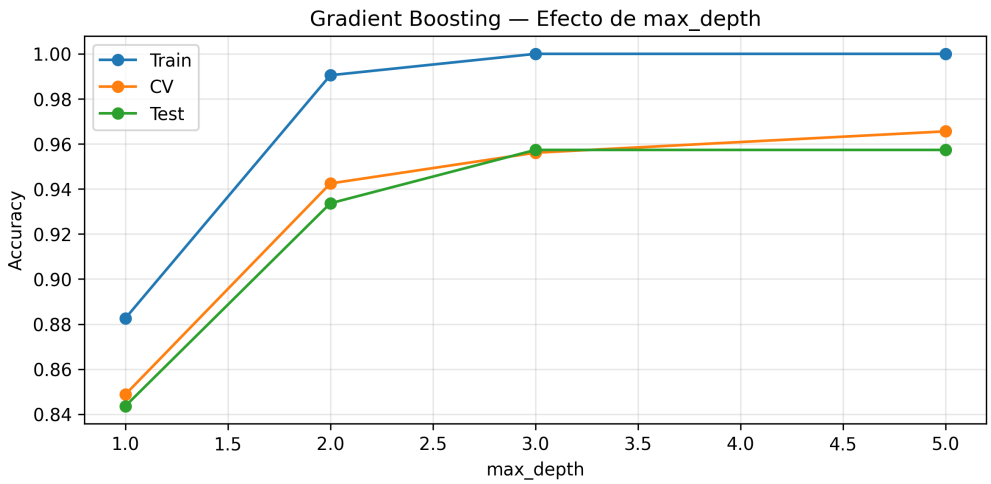

Figura 10: Efecto de la profundidad de los árboles base en Gradient Boosting.

En esta ejecución, `max_depth=5` pasa a ser la mejor opción en validación cruzada, mientras que en test empata prácticamente con `max_depth=3`. La diferencia no es grande, pero sí suficiente para mostrar que, en este problema, árboles base algo más profundos pueden capturar interacciones útiles sin degradar la generalización. En cambio, los stumps siguen quedando muy lejos, lo que confirma que aquí hacen falta interacciones entre varias variables.

6.4. GridSearch de Gradient Boosting

Tabla 25: Mejores configuraciones encontradas para Gradient Boosting.

n_estimators	learning_rate	max_depth	subsample	CV Acc	Test Acc
200	0.1000	3	1.0000	0.9620	0.9739
200	0.1000	3	0.8000	0.9614	0.9621
100	0.5000	3	0.8000	0.9614	0.9621
200	0.5000	3	1.0000	0.9591	0.9573
200	0.5000	3	0.8000	0.9585	0.9645
200	0.0500	3	1.0000	0.9585	0.9550
100	0.5000	3	1.0000	0.9567	0.9645
100	0.1000	3	0.8000	0.9561	0.9479
100	0.1000	3	1.0000	0.9561	0.9573
200	0.0500	3	0.8000	0.9555	0.9479

El mejor modelo seleccionado por `GridSearchCV` es ahora `learning_rate = 0.1`, `max_depth = 3`, `n_estimators = 200` y `subsample = 1.0`, con **0.9620** de CV y **0.9739** en test.

Este resultado es especialmente satisfactorio desde el punto de vista metodológico, porque en esta nueva ejecución ya no existe discrepancia entre la mejor configuración por validación cruzada y la mejor observada en test. Ambas lecturas apuntan a la misma receta clásica de boosting: un `learning_rate` moderado, suficientes iteraciones y árboles base de profundidad intermedia.

6.5. AdaBoost con stumps

Tabla 26: Resultados de AdaBoost usando stumps como estimadores base.

n_estimators	learning_rate	CV Accuracy	Test Accuracy	Tiempo (s)
200	0.1000	0.4555	0.4313	0.4120
100	0.5000	0.4549	0.4597	0.2144
100	1.0000	0.4484	0.4242	0.2093
50	1.0000	0.4407	0.5355	0.1069

AdaBoost sigue siendo, con mucha diferencia, el peor modelo de la práctica. Las accuracies en validación cruzada rondan el **44–46 %** y el modelo seleccionado por CV cae hasta **43.13 %** en test. Hay un caso con **53.55 %** en test, pero tampoco cambia la conclusión de fondo: los stumps son demasiado débiles para un problema multiclase de este nivel.

6.6. Resumen de boosting

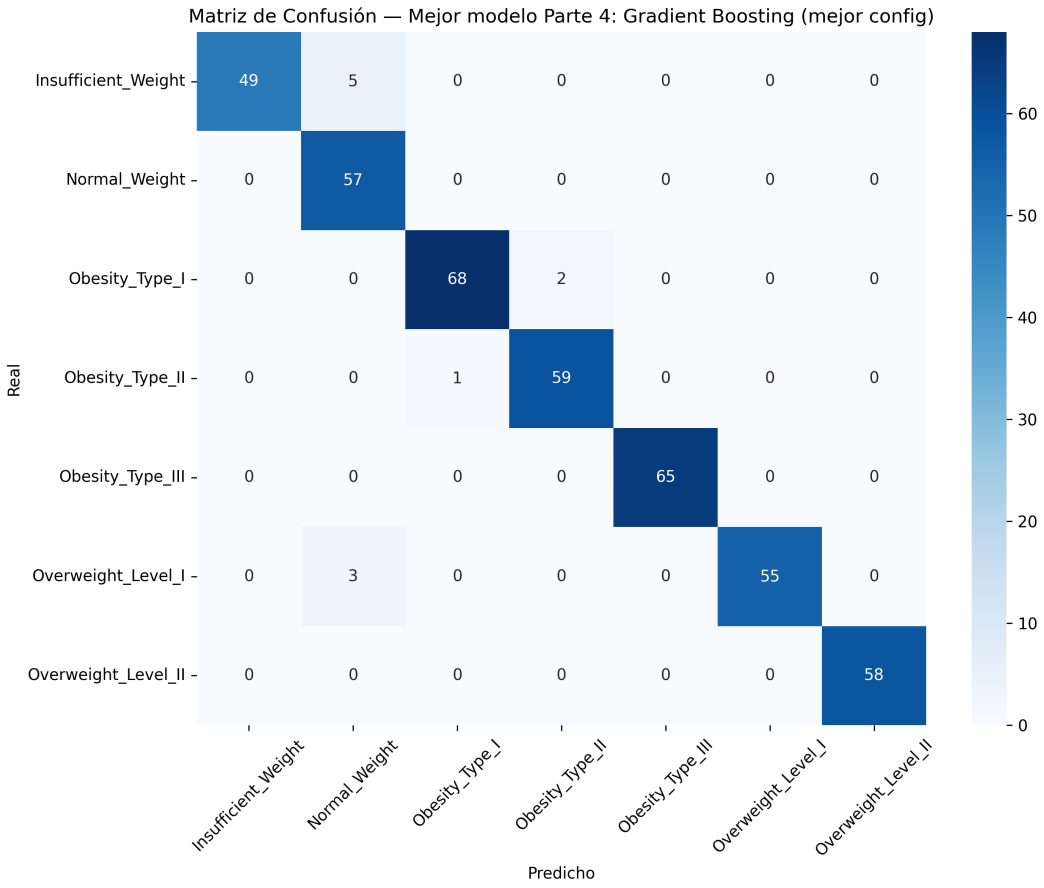

Figura 11: Matriz de confusión del mejor modelo de boosting seleccionado por el notebook.

Tabla 27: Comparativa final entre Gradient Boosting, Random Forest y AdaBoost.

Modelo	Test Accuracy	Precision (w)	Recall (w)	F1 (w)
Gradient Boosting (mejor config)	0.9739	0.9763	0.9739	0.9742
Gradient Boosting (default)	0.9573	0.9612	0.9573	0.9579
Random Forest (mejor config)	0.9573	0.9589	0.9573	0.9576
AdaBoost (mejor config)	0.4313	0.2492	0.4313	0.3116

Gradient Boosting sigue siendo la familia más fuerte del bloque. En esta nueva ejecución, además, el modelo seleccionado por validación cruzada alcanza directamente **97.39 %** en test, por encima del mejor Random Forest y muy por encima de AdaBoost. El hecho de que el mejor modelo por CV coincida también con el mejor test observado refuerza aún más la confianza en esta familia.

7. Comparación Global

Tabla 28: Comparativa global de los modelos seleccionados automáticamente por el notebook para cada familia.

Modelo	CV Accuracy	Test Accuracy	Precision (w)	Recall (w)	F1 (w)	Tiempo (s)
Gradient Boosting (best)	0.9620	0.9739	0.9763	0.9739	0.9742	4.3849
Random Forest (best)	0.9561	0.9573	0.9589	0.9573	0.9576	0.2079
SVM (best RBF)	0.9401	0.9408	0.9417	0.9408	0.9405	0.0430
Decision Tree (best pre-pruning)	0.9383	0.9265	0.9295	0.9265	0.9271	0.0074
AdaBoost (best)	0.4555	0.4313	0.2492	0.4313	0.3116	0.4167

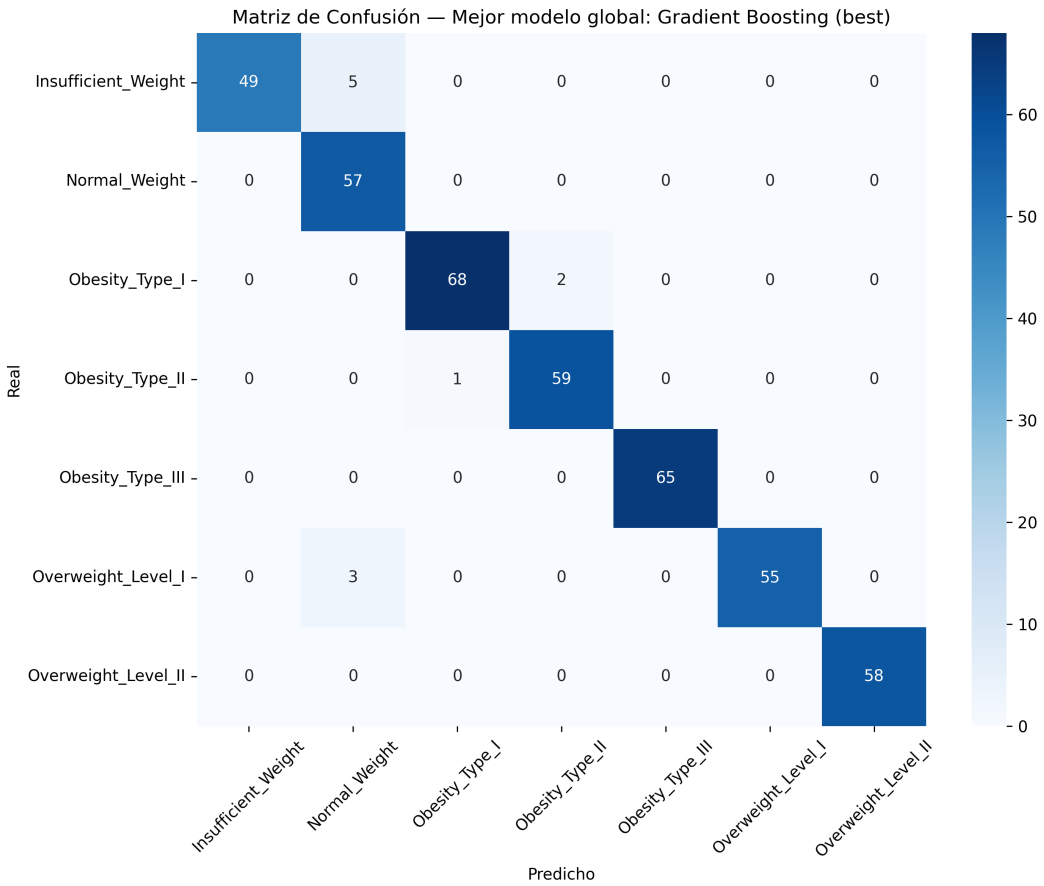

Figura 12: Matriz de confusión del mejor modelo global según la comparativa automática del notebook.

Siguiendo la selección final realizada por el notebook, el ranking queda así:

$$\text{Gradient Boosting} > \text{Random Forest} > \text{SVM-RBF} > \text{Decision Tree} \gg \text{AdaBoost}$$

En cifras:

- **Gradient Boosting**: **0.9739** accuracy, **0.9742** F1, tiempo **4.38 s**
- **Random Forest**: **0.9573** accuracy, **0.9576** F1, tiempo **0.21 s**
- **SVM-RBF**: **0.9408** accuracy, **0.9405** F1, tiempo **0.04 s**
- **Decision Tree** (seleccionado por el notebook): **0.9265** accuracy, **0.9271** F1
- **AdaBoost**: **0.4313** accuracy, **0.3116** F1, tiempo **0.42 s**

Sin embargo, esta comparativa automática no agota toda la información del notebook. De hecho, dos resultados aislados del propio experimento son aún más llamativos:

- El árbol con criterio `entropy/log_loss` alcanza **0.9739** en test.
- El mejor Gradient Boosting del GridSearch, con `learning_rate=0.1`, `max_depth=3`, `n_estimators=200` y `subsample=1.0`, también alcanza **0.9739** en test.

Esto obliga a distinguir entre "mejor modelo seleccionado automáticamente por el notebook" y "mejor resultado observado dentro de los experimentos ejecutados".

8. Discusión

8.1. Qué cambia al corregir el leakage

La corrección metodológica en Box-Cox no solo era necesaria por limpieza experimental, sino que además cambia de forma perceptible la lectura de los resultados. SVM se vuelve más lineal de lo esperado, Random Forest queda algo mejor posicionado, y los árboles muestran un rendimiento con entropía extraordinariamente alto. Es decir, la fuga de información no estaba alterando todo el ranking de manera dramática, pero sí modificaba algunos matices importantes de interpretación.

8.2. Por qué SVM funciona tan bien

Que SVM lineal y SVM-RBF queden tan cerca entre sí, e incluso con ligera ventaja del lineal, es una pista muy potente sobre la estructura del problema. No estamos ante un dataset caótico ni ante una geometría altamente intrincada. El encoding y la estandarización ya colocan los datos en un espacio donde una frontera casi lineal funciona sorprendentemente bien.

8.3. El papel de Weight y Height

Todas las familias de modelos convergen en la misma conclusión: `Weight` y `Height` siguen siendo el núcleo del problema. No es una casualidad estadística, sino la traducción algorítmica de una relación fisiológica evidente: el nivel de obesidad depende en gran medida del IMC, y el IMC está determinado esencialmente por peso y altura.

8.4. Lectura crítica del notebook

La revisión de la nueva ejecución deja dos observaciones especialmente importantes:

1. El árbol seleccionado en la comparativa final automática no representa el mejor resultado real de la familia de árboles, porque la comparación de criterios muestra un rendimiento muy superior con `entropy/log_loss`.

2. En Gradient Boosting, el mejor estimador por validación cruzada no coincide exactamente con la configuración que obtiene el mejor test en esta partición. Esto no invalida el uso de CV; simplemente recuerda que una partición concreta puede favorecer ligeramente a otra combinación vecina.

9. Conclusiones

1. **Gradient Boosting sigue siendo el mejor modelo dentro de la comparativa global automática del notebook**. La configuración seleccionada por validación cruzada alcanza **97.39 %** de accuracy y **0.9742** de F1 ponderado en test.

2. **Random Forest ofrece un equilibrio rendimiento-coste excelente**. Se queda muy cerca del mejor boosting con **95.73 %** de accuracy y entrena mucho más rápido.

3. **SVM confirma que el problema es bastante separable**. Tras eliminar el leakage, el kernel lineal pasa a ser ligeramente mejor que el RBF optimizado, señal de que la no linealidad existe pero no es dominante.

4. **Los árboles individuales merecen mucha más atención de la que sugiere la comparativa final automática**. El experimento con `entropy/log_loss` alcanza **97.39 %** en test, un resultado sobresaliente para un modelo interpretable.

5. **AdaBoost con stumps no es adecuado para este problema**. El cuello de botella no está en el número de iteraciones, sino en la debilidad excesiva del clasificador base para una tarea multiclase de este nivel.

6. **La corrección del leakage era necesaria y cambia la historia experimental**. No altera la idea general de que los ensembles son muy fuertes, pero sí modifica conclusiones finas sobre qué familia domina realmente y hasta qué punto el problema requiere no linealidad.

En conjunto, la práctica muestra con bastante claridad cómo cambia el rendimiento al pasar de modelos interpretables pero frágiles a ensembles cada vez más sofisticados. Al mismo tiempo, esta versión corregida deja una enseñanza metodológica igual de importante: en aprendizaje automático no basta con obtener buenos números; también hay que asegurarse de que el pipeline que los produce es estadísticamente correcto.